```
        {
            var ipAddress = context.Connection.RemoteIpAddress;
            logger.LogInformation($"Visitor is from {ipAddress}");  ❸
            await _next(context);                                   ❹
        }
    }
}
```

❶ The constructor takes a RequestDelegate as a parameter.

❷ The InvokeAsync method returns a task and has an HttpContext as a first parameter. Any additional services are injected into the Invoke/InvokeAsync method.

❸ Performs the processing within the InvokeAsync method

❹ Passes control to the next middleware in the pipeline

Next, add a `using` directive to the top of the Program.cs file to bring the `WebApplication1` namespace into scope:

```
using WebApplication1;
var builder = WebApplication.CreateBuilder(args);
```

The middleware class is added to the pipeline via the `UseMiddleware` method on the `WebApplication`. There are two versions of this method. The first takes a type as a parameter:

```
app.UseMiddleware(typeof(IpAddressMiddleware));
```

The second version takes a generic parameter, representing the middleware class. This version is the one you are more likely to encounter:

```
app.UseMiddleware<IpAddressMiddleware>();
```

Alternatively, it is recommended that you create your own extension method on `IApplicationBuilder` to register your middleware. This example, shown in the following listing, is placed in a class named `Extensions`, which has also been added to the root of the project.